Guía de Práctica: Estructura Frontend y Enrutamiento para Catálogo Multinivel

Esta guía técnica describe el proceso de implementación de la arquitectura base para una aplicación de catálogo multinivel. Se enfoca en la gestión de rutas dinámicas y la creación de interfaces modulares utilizando React y Tailwind CSS.

1. Objetivos

- Establecer un sistema de enrutamiento dinámico profesional utilizando la librería React Router.
- Construir una arquitectura de componentes basada en un Layout principal que optimice la navegación.
- Desarrollar interfaces de usuario base para el catálogo utilizando datos estáticos (hardcodeados) que sirvan como prototipo funcional para la futura integración de APIs.
- Garantizar un diseño responsivo y estético mediante el uso de utilidades de Tailwind CSS.

2. Configuración de Rutas

Para comenzar con la implementación del enrutamiento, es necesario integrar la librería estándar de la industria para aplicaciones React.

Ejecute el siguiente comando en la terminal de su proyecto:

npm install react-router-dom

Asegúrese de verificar que la dependencia haya sido añadida correctamente en el archivo `package.json`.

3. Componentes

Navbar

```
import { Link, useNavigate } from "react-router-dom";
import { useCart } from "../context/CartContext";
import { useAuth } from "../context/AuthContext";

const Navbar = () => {
  const { totalItems } = useCart();
  const { logout, userEmail } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate("/login");
  };

  return (
    <header className="h-16 bg-white border-b border-slate-200 flex items-
center justify-between px-8">
      <h2 className="text-slate-600 font-medium text-lg">
        Panel de Administración
      </h2>
      <div className="flex items-end gap-6">
        <Link
          to="/carrito"
          className="relative p-2 hover:bg-slate-100 rounded-full transition"
        >
          <span className="text-xl">🛒</span>
          {totalItems > 0 && (
            <span className="absolute top-0 right-0 bg-indigo-600 text-white
text-xs font-bold w-5 h-5 flex items-center justify-center rounded-full
transform translate-x-1 -translate-y-1">
              {totalItems}
            </span>
```

```
        )}
      </Link>

      <div className="flex items-center gap-4">
        <span className="text-sm text-slate-500">{userEmail}</span>

        {/* Contenedor relativo con la clase 'group' para detectar el hover */}
        <div className="relative group cursor-pointer pb-2">
          {/* Círculo del usuario / Avatar */}
          <div className="w-8 h-8 rounded-full bg-slate-200 overflow-hidden
border border-slate-300 flex items-center justify-center">
            {/*
      NOTA PARA LA API:
      Aquí reemplazarás el 'src' quemado por la variable de tu estado,
      por ejemplo: src={userAvatar || defaultImage}
    */}
            <img

src="https://fastly.picsum.photos/id/64/4326/2884.jpg?hmac=9_SzX666YRpR
_fOyYStXpfSiJ_edO3ghlSRnH2w09Kg"
              alt="Avatar del usuario"
              className="w-full h-full object-cover"
            />
          </div>

          {/* Menú desplegable */}
          <div className="absolute right-0 top-full mt-1 w-36 bg-white border
border-slate-200 rounded-lg shadow-lg opacity-0 invisible group-
hover:opacity-100 group-hover:visible transition-all duration-200 z-50">
            <button
              onClick={handleLogout}
              className="w-full text-left px-4 py-2 text-sm text-red-600 font-
semibold hover:bg-red-50 rounded-md transition-colors"
            >
              Cerrar Sesión
            </button>
          </div>
        </div>
```

```
      </div>
    </div>
  </header>
 );
};

export default Navbar;
```

Sidebar

```
import { Link } from "react-router-dom";

const Sidebar = () => {
 return (
  <aside className="w-64 bg-slate-900 text-white flex flex-col">
   <div className="p-6 text-2xl font-bold border-b border-slate-700">
    MultiCatálogo
   </div>

   <nav className="flex-1 p-4 space-y-2">
    <Link
     to="/"
     className="block p-3 rounded hover:bg-slate-800 transition"
    >
     Dashboard
    </Link>

    <Link
     to="/catalogo"
     className="block p-3 rounded hover:bg-slate-800 transition"
    >
     Catálogo
    </Link>

    <Link
     to="/mi-red"
```

```
          className="block p-3 rounded hover:bg-slate-800 transition"
        >
          Mi Red
        </Link>
      </nav>
    </aside>
  );
};

export default Sidebar;
```

Componente Layout

El componente Layout servirá como el contenedor estructural de nuestra aplicación, manteniendo elementos persistentes como la barra de navegación y el menú lateral.

Cree un archivo denominado `Layout.jsx`:

```
import { Outlet } from "react-router-dom";
import Sidebar from "./Sidebar";
import Navbar from "./Navbar";

const Layout = () => {
return (
    <div className="flex h-screen bg-slate-50">
      <Sidebar />

      {/* Área de Contenido Principal */}
      <div className="flex-1 flex flex-col overflow-hidden">
        <Navbar />
        <main className="flex-1 overflow-x-hidden overflow-y-auto p-8">
<Outlet />
        </main>
      </div>
    </div>
  );
};
```

export default Layout;

Catálogo Multinivel

A continuación, se definen las tres vistas principales solicitadas. Estas utilizan datos locales para simular la experiencia del usuario final.

Dashboard.jsx

Resumen ejecutivo con indicadores clave.

```
const Dashboard = () => {
  return (
    <div>
      <h1 className="text-2xl font-bold text-slate-800 mb-6">
        Resumen General
      </h1>
      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        <div className="bg-white p-6 rounded-lg shadow-sm border border-
slate-100">
          <p className="text-sm text-slate-500 uppercase font-semibold">
            Ventas Totales
          </p>
          <p className="text-3xl font-bold text-indigo-600">$12,450.00</p>
        </div>
        <div className="bg-white p-6 rounded-lg shadow-sm border border-
slate-100">
          <p className="text-sm text-slate-500 uppercase font-semibold">
            Referidos Activos
          </p>
          <p className="text-3xl font-bold text-indigo-600">24</p>
        </div>
        <div className="bg-white p-6 rounded-lg shadow-sm border border-
slate-100">
          <p className="text-sm text-slate-500 uppercase font-semibold">
            Nivel Actual
          </p>
          <p className="text-3xl font-bold text-indigo-600">Diamante</p>
        </div>
```

```
      </div>
    </div>
  );
};
```

export default Dashboard;

Catalogo

Visualización de productos en formato de cuadrícula.

```
// src/components/Catalogo.tsx
import { useCart } from '../context/CartContext';

const Catalogo = () => {
  const { addToCart } = useCart(); // <-- Usamos la función del contexto

  const productos = [
    { id: 1, nombre: "Serum Revitalizante", precio: 45.00, img:
"https://picsum.photos/seed/serum/150" },
    { id: 2, nombre: "Crema Hidratante Pro", precio: 32.50, img:
"https://picsum.photos/seed/crema/150" },
    { id: 3, nombre: "Tónico Purificante", precio: 28.00, img:
"https://picsum.photos/seed/tonico/150" },
    { id: 4, nombre: "Mascarilla Nocturna", precio: 50.00, img:
"https://picsum.photos/seed/mascarilla/150" },
  ];

  return (
    <div>
      <h1 className="text-2xl font-bold text-slate-800 mb-6">Catálogo de
Productos</h1>
```

```
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6">
        {productos.map((prod) => (
          <div key={prod.id} className="bg-white rounded-lg overflow-hidden
border border-slate-200 shadow-sm flex flex-col">
            <img src={prod.img} alt={prod.nombre} className="w-full h-40
object-cover" />
            <div className="p-4 flex flex-col flex-1">
              <h3 className="font-semibold text-slate-700">{prod.nombre}</h3>
              <p className="text-indigo-600 font-bold mt-2 mb-
4">${prod.precio.toFixed(2)}</p>

              <button
                onClick={() => addToCart(prod)}
                className="mt-auto w-full bg-slate-900 text-white py-2 rounded
text-sm hover:bg-indigo-600 transition"
              >
                Añadir al Carrito
              </button>
            </div>
          </div>
        ))}
      </div>
    </div>
  );
};

export default Catalogo;
```

MiRed

Estructura de red o lista de referidos.

```
const MiRed = () => {
  const referidos = [
    { id: 1, nombre: "Ana García", nivel: "Nivel 1", ventas: "$1,200" },
    { id: 2, nombre: "Luis Poveda", nivel: "Nivel 1", ventas: "$850" },
    { id: 3, nombre: "Marta Sánchez", nivel: "Nivel 2", ventas: "$430" },
  ];
```

```
  return (
    <div>
      <h1 className="text-2xl font-bold text-slate-800 mb-6">
        Mi Red de Referidos
      </h1>

      <div className="bg-white rounded-lg border border-slate-200 shadow-
sm overflow-hidden">
        <table className="w-full text-left">
          <thead className="bg-slate-50 border-b border-slate-200">
            <tr>
              <th className="p-4 font-semibold text-slate-600">Nombre</th>
              <th className="p-4 font-semibold text-slate-600">Jerarquía</th>
              <th className="p-4 font-semibold text-slate-600">
                Ventas Mensuales
              </th>
            </tr>
          </thead>

          <tbody>
            {referidos.map((ref) => (
              <tr
                key={ref.id}
                className="border-b border-slate-100 hover:bg-slate-50"
              >
                <td className="p-4 text-slate-700">{ref.nombre}</td>
                <td className="p-4 text-slate-500">{ref.nivel}</td>
                <td className="p-4 text-indigo-600 font-medium">
                  {ref.ventas}
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
```

```
};

export default MiRed;
```

Carrito

```
// src/components/Carrito.tsx
import { useCart } from '../context/CartContext';

const Carrito = () => {
  const { cart, removeFromCart, totalPrice } = useCart();

  return (
    <div>
      <h1 className="text-2xl font-bold text-slate-800 mb-6">Tu Carrito de
Compras</h1>

      {cart.length === 0 ? (
        <div className="bg-white p-8 rounded-lg border border-slate-200
shadow-sm text-center">
          <p className="text-slate-500">Tu carrito está vacío actualmente.</p>
        </div>
      ) : (
        <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
          {/* Lista de productos */}
          <div className="lg:col-span-2 space-y-4">
            {cart.map((item) => (
              <div key={item.id} className="flex items-center justify-between bg-
white p-4 rounded-lg border border-slate-200 shadow-sm">
                <div className="flex items-center gap-4">
                  <img src={item.img} alt={item.nombre} className="w-16 h-16
object-cover rounded-md" />
                  <div>
                    <h3 className="font-semibold text-slate-
800">{item.nombre}</h3>
                    <p className="text-sm text-slate-500">Cantidad:
{item.cantidad}</p>
```

```
              </div>
             </div>
             <div className="flex items-center gap-6">
              <p className="font-bold text-indigo-600">${(item.precio *
item.cantidad).toFixed(2)}</p>
              <button
               onClick={() => removeFromCart(item.id)}
               className="text-red-500 hover:text-red-700 transition"
               title="Eliminar producto"
              >
               🗑
              </button>
             </div>
            </div>
           ))}
          </div>

          {/* Resumen de pago */}
          <div className="bg-white p-6 rounded-lg border border-slate-200
shadow-sm h-fit">
           <h2 className="text-lg font-bold text-slate-800 mb-4">Resumen del
Pedido</h2>
           <div className="flex justify-between border-b border-slate-100 pb-4
mb-4">
            <span className="text-slate-600">Subtotal</span>
            <span className="font-semibold">${totalPrice.toFixed(2)}</span>
           </div>
           <div className="flex justify-between items-center mb-6">
            <span className="text-slate-800 font-bold">Total a Pagar</span>
            <span className="text-2xl font-bold text-indigo-
600">${totalPrice.toFixed(2)}</span>
           </div>
           <button className="w-full bg-indigo-600 text-white py-3 rounded-lg
font-semibold hover:bg-indigo-700 transition">
            Proceder al Pago
           </button>
          </div>
         </div>
```

```
      )}
    </div>
  );
};

export default Carrito;
```

Autenticación

A la misma altura de la carpeta components creamos la carpeta **context**, y dentro de la carpeta context añadimos dos archivos **AuthContext.tsx** y **CartContext.tsx**

AuthContext

```
// src/context/AuthContext.tsx
import { createContext, useContext, useState, type ReactNode } from 'react';

interface AuthContextType {
  isAuthenticated: boolean;
  userEmail: string | null; // <-- 1. Nuevo estado para el correo
  login: (email: string) => void; // <-- 2. La función ahora recibe el correo
  logout: () => void;
}

const AuthContext = createContext<AuthContextType |
undefined>(undefined);

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth debe ser usado dentro de un AuthProvider");
  }
  return context;
};
```

```
interface AuthProviderProps {
  children: ReactNode;
}

export const AuthProvider = ({ children }: AuthProviderProps) => {
  const [isAuthenticated, setIsAuthenticated] = useState<boolean>(false);
  const [userEmail, setUserEmail] = useState<string | null>(null); // <-- 3.
Estado local

  // 4. Actualizamos las funciones
  const login = (email: string) => {
    setIsAuthenticated(true);
    setUserEmail(email); // Guardamos el correo
  };

  const logout = () => {
    setIsAuthenticated(false);
    setUserEmail(null); // Limpiamos el correo al salir
  };

  return (
    <AuthContext.Provider value={{ isAuthenticated, userEmail, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
```

CartContext

```
// src/context/CartContext.tsx
import { createContext, useContext, useState, type ReactNode } from 'react';

// 1. Definimos las interfaces (tipos de datos)
export interface Producto {
  id: number;
  nombre: string;
  precio: number;
  img: string;
```

```
}

export interface CartItem extends Producto {
  cantidad: number;
}

interface CartContextType {
  cart: CartItem[];
  addToCart: (producto: Producto) => void;
  removeFromCart: (id: number) => void;
  totalItems: number;
  totalPrice: number;
}

// 2. Creamos el contexto indicando que puede ser CartContextType o
undefined
const CartContext = createContext<CartContextType | undefined>(undefined);

// 3. Hook personalizado con validación de tipo
export const useCart = () => {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error("useCart debe ser usado dentro de un CartProvider");
  }
  return context;
};

// 4. Tipamos los props del Provider
interface CartProviderProps {
  children: ReactNode;
}

// 5. El Provider
export const CartProvider = ({ children }: CartProviderProps) => {
  // Le decimos a useState que este arreglo contendrá objetos de tipo CartItem
  const [cart, setCart] = useState<CartItem[]>([]);

  const addToCart = (producto: Producto) => {
```

```
    setCart((prevCart) => {
      const itemExists = prevCart.find((item) => item.id === producto.id);
      if (itemExists) {
        return prevCart.map((item) =>
          item.id === producto.id
            ? { ...item, cantidad: item.cantidad + 1 }
            : item
        );
      }
      return [...prevCart, { ...producto, cantidad: 1 }];
    });
  };

  const removeFromCart = (id: number) => {
    setCart((prevCart) => prevCart.filter((item) => item.id !== id));
  };

  const totalItems = cart.reduce((total, item) => total + item.cantidad, 0);
  const totalPrice = cart.reduce((total, item) => total + item.precio *
item.cantidad, 0);

  return (
    <CartContext.Provider value={{ cart, addToCart, removeFromCart,
totalItems, totalPrice }}>
      {children}
    </CartContext.Provider>
  );
};
```

Login

```
// src/components/Login.tsx
import { useState, type FormEvent } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const Login = () => {
  const [email, setEmail] = useState<string>('');
```

```
  const [password, setPassword] = useState<string>('');
  const [error, setError] = useState<string>('');

  const navigate = useNavigate();
  const { login } = useAuth();

  const handleSubmit = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();

    // Simulación de validación hardcodeada (A futuro se reemplazará por
llamada a API)
    if (email === 'admin@upse.edu.ec' && password === '123456') {
      setError('');
      login(email); // Cambiamos el estado global a autenticado
      navigate('/'); // Redirigimos al Dashboard
    } else {
      setError('Credenciales incorrectas. Usa admin@upse.edu.ec / 123456');
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-100
px-4">
      <div className="max-w-md w-full bg-white rounded-xl shadow-md p-8
border border-slate-200">
        <div className="text-center mb-8">
          <h2 className="text-3xl font-bold text-slate-900">MultiCatálogo</h2>
          <p className="text-slate-500 mt-2">Ingresa a tu cuenta para
continuar</p>
        </div>

        {error && (
          <div className="bg-red-50 text-red-600 p-3 rounded-lg text-sm mb-6
text-center border border-red-200">
            {error}
          </div>
        )}
```

```
      <form onSubmit={handleSubmit} className="space-y-6">
        <div>
          <label className="block text-sm font-medium text-slate-700 mb-
2">Correo Electrónico</label>
          <input
            type="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            className="w-full px-4 py-3 rounded-lg border border-slate-300
focus:ring-2 focus:ring-indigo-600 focus:border-indigo-600 outline-none
transition"
            placeholder="admin@upse.edu.ec"
            required
          />
        </div>

        <div>
          <label className="block text-sm font-medium text-slate-700 mb-
2">Contraseña</label>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            className="w-full px-4 py-3 rounded-lg border border-slate-300
focus:ring-2 focus:ring-indigo-600 focus:border-indigo-600 outline-none
transition"
            placeholder="••••••"
            required
          />
        </div>

        <button
          type="submit"
          className="w-full bg-indigo-600 text-white font-bold py-3 rounded-lg
hover:bg-indigo-700 transition"
        >
          Iniciar Sesión
        </button>
```

```
      </form>
    </div>
  </div>
 );
};

export default Login;
```

Integración en App.tsx

Finalmente, orqueste todas las piezas en el punto de entrada principal utilizando los componentes de enrutamiento.

```
// src/App.tsx
import { BrowserRouter, Routes, Route, Navigate, Outlet } from 'react-router-dom';
import Layout from './components/Layout';
import Dashboard from './components/Dashboard';
import Catalogo from './components/Catalogo';
import MiRed from './components/MiRed';
import Carrito from './components/Carrito';
import Login from './components/Login';
import { CartProvider } from './context/CartContext';
import { AuthProvider, useAuth } from './context/AuthContext';

// Componente para proteger las rutas privadas
const ProtectedRoute = () => {
 const { isAuthenticated } = useAuth();

 // Si no está autenticado, lo enviamos al login
 if (!isAuthenticated) {
  return <Navigate to="/login" replace />;
 }

 // Si está autenticado, renderiza las rutas hijas (Outlet)
 return <Outlet />;
};
```

```
function App() {
  return (
    <AuthProvider> {/* Proveedor de Autenticación */}
      <CartProvider> {/* Proveedor del Carrito */}
        <BrowserRouter>
          <Routes>
            {/* Ruta pública */}
            <Route path="/login" element={<Login />} />

            {/* Rutas protegidas */}
            <Route element={<ProtectedRoute />}>
              <Route path="/" element={<Layout />}>
                <Route index element={<Dashboard />} />
                <Route path="catalogo" element={<Catalogo />} />
                <Route path="mi-red" element={<MiRed />} />
                <Route path="carrito" element={<Carrito />} />
              </Route>
            </Route>

            {/* Ruta comodín para capturar 404 y redirigir */}
            <Route path="*" element={<Navigate to="/" replace />} />
          </Routes>
        </BrowserRouter>
      </CartProvider>
    </AuthProvider>
  );
}

export default App;
```

Ejercicio de Extensión

Modifique el componente `Sidebar` para que acepte una prop llamada `isCollapsed` (booleano). Si es true, el sidebar debe reducir su ancho a 80px y mostrar solo los iconos. Implemente un botón de "Toggle" en el `Navbar` que controle este estado global.

Nota: Debe agregar iconos a las opciones de menú.